

# ComeBack: Assessing LLM Decompilation for Go

Matthew Cai

UC Berkeley

matthew\_cai@berkeley.edu

Jonah Bedouch

UC Berkeley

jonahbedouch@berkeley.edu

**Abstract**—Recent advancements in the quality of state-of-the-art (SOTA) LLMs have yielded promising results across a myriad of fields. In response to this, recent works have attempted to improve the quality of decompilation results by allowing LLMs to refine classical decompilations provided by Ghidra. However, this technique has been almost exclusively explored in C, not in high-level programming languages, which pose many unique challenges to traditional decompilation, including large, statically linked standard libraries and language runtimes. Go is a particularly important target: it dominates cloud-native infrastructure and has seen rapid adoption among malware authors, making reverse engineering Go binaries an increasingly critical task.

This paper provides two main contributions. First, we assemble a benchmark consisting of the source code, binaries, and Ghidra decompilations of popular Go projects. Next, we implement a system that prompts a SOTA model to recover the original Golang based on the Ghidra decompilations, and evaluate the quality of the recovered output using CodeBLEU, LLM-as-a-Judge, and syntax validity metrics.

## I. INTRODUCTION

*Decompilation* is the process of converting a low-level binary (consisting of assembly instructions) into a higher-level intermediary language, in order to enable analysis of the underlying binary without possessing the source. Decompilation is commonly used in reverse engineering to port or update code whose source has been lost, to better understand the implementation of closed-source programs, and to perform security and malware analysis on programs whose source is not generally available.

Compilation is a lossy, many-to-one transformation: variable names, type annotations, and high-level control structure are discarded, making inversion inherently ambiguous and difficult. Hence, decompilation has been extensively analyzed by both industry and academia in past decades. Classical decompilation techniques rely on complex control-flow analysis, structural analysis, and a large number of hand-specified heuristics in order to convert a binary into an Intermediate Representation (IR), then reconstruct a low-level language, such as C, from this IR [1]. While SOTA classical decompilers can analyze primitive and pointer-based types [2], the decompilation process is highly complex, and often produces output that is difficult to analyze for large programs written in C. Additionally, classical decompilers such as Ghidra [3] or IDA Pro [4] often produce significantly degraded decompilations, with regards to readability, for binaries produced by higher-level programming languages that are used for many modern closed-source programs. This is a direct consequence of the presence of large standard libraries, complex dynamic dispatch tables, large pointer-based data structures, unconven-

tional calling conventions, and extensive language runtimes, which add significant complexity and noise to the binary. And, regardless, the decompilation result is ultimately a pseudo-C representation which is challenging to read or analyze, as it is inherently structurally and semantically dissimilar from the source code of the original language.

Due to the above limitations, recent literature has attempted to iterate upon classical decompilation through the use of Large-Language Models (LLMs). Various approaches have been explored, which largely fall into two major techniques. Many systems sidestep the traditional decompilation step entirely, relying on fine-tuned models combined with unique techniques to achieve *end-to-end* decompilation ([5], [6], [7], [8], [9], [10]). Meanwhile, another set of systems rely on existing general-purpose models in order to *refine* the output of classical decompilers, transforming convoluted, decompiled C code into a more human-readable representation suitable for reverse engineering analysis ([5], [11], [12], [13]).

In general, however, these LLM-driven techniques focus primarily on decompiling large binaries originally written in C into a more human-readable C than the code output by classical decompilers. Little effort has been placed into exploring the ability of LLMs to lift a binary compiled from a higher-level language back into the syntax of the original language.

Google’s Go Programming Language [14] is a particularly interesting target for such an endeavor. Go has become the predominant language for cloud-native infrastructure, with over 75% of projects in the Cloud Native Computing Foundation being written in Go [15]. Moreover, it has grown increasingly popular among malware authors due to its first-class cross-compilation support; its ability to produce statically linked, self-contained binaries without external dependencies; and the simple fact that Go binaries are much more challenging to reverse engineer, particularly when combined with Go obfuscation tools [16]. In fact, Unit 42 of Palo Alto Networks reported a 2000% increase in identified Go malware samples from 2017 to 2019 [17], while a study by CrowdStrike revealed an 80% increase in Go malware samples in just three months, from June to August 2021 [18]. Consequently, reverse engineers have been increasingly encountering binaries compiled from Go.

This paper serves as an initial exploration into the use of LLM-enhanced decompilation to recover source code written in a fully-featured high-level language, Go. We present *ComeBack*, a refinement-based LLM decompilation suite designed to convert binaries compiled from Go into their original, high-level Go code, by first using Ghidra to produce C-level decompilations, then refining these compilations through the

use of a SOTA LLM. Additionally, we construct a benchmark consisting of some of the most used Go programs on GitHub, and evaluate our system against this benchmark, assessing for semantic similarity as our primary measure of success.

The remainder of this paper will be structured as follows. In Section II, we discuss related works and their applicability to ComeBack. In Section III, we discuss the process by which we created a dataset in order to audit the effectiveness of ComeBack and our approach for the design and implementation of ComeBack, including the decisions (and constraints) that influenced the final design. In Section IV, we discuss the results of running ComeBack on our dataset, including qualitative analysis of code readability and quantitative similarity metrics. Finally, we discuss limitations and future directions of exploration in Section V, before concluding in Section VI.

## II. RELATED WORKS

Much of the existing decompilation literature has focused on enhancing classical and LLM-based methods for decompiling programs originally compiled in C. In this section, we discuss some noteworthy standouts in the realm of LLM-based C decompilation, a few interesting works that discuss decompiling languages outside of C, and seminal decompilation benchmarks. These serve to contextualize our work in Golang decompilation.

We acknowledge the existence of a substantial corpus of work around the decompilation of Java (such as [19]), but choose to omit detailed discussion around this because decompiling Java presents a fundamentally different problem than other programming languages. Java Bytecode is specifically optimized to run in the Java Virtual Machine (JVM), and thus retains a significantly greater portion of its original semantics (including type information). This enables comparatively effective recovery (i.e. 78% semantically equivalent code output relying solely on classical methods in [19]).

### A. LLM-Based C Decompilation Techniques

LLM4Decompile [5] introduced the first open-source LLM series fine-tuned for end-to-end decompilation, known as LLM4Decompile-End, which translates assembly code directly to C. ReF Decompile [6] proposed augmentations to the end-to-end pipeline: relabeling jump targets with labels to improve control flow recovery and separately inferring type information. SALT4Decompile [9] constructs source-level abstract logic trees (SALTs) from binary to approximate higher-level control flow, and fine-tuned an LLM on these representations. SK2Decompile [8] decomposes the problem into two sequential phases, structure recovery and identifier naming, and fine-tunes a distinct model for each via reinforcement learning, enabling independent optimization of the control flow correctness and code readability.

Meanwhile, a parallel line of work instead refines the pseudo-C output of classical decompilers. LLM4Decompile also offers a refinement LLM series, LLM4Decompile-Ref, that takes Ghidra’s output and returns improved C code. Context-Guided Decompilation [11] leveraged in-context learning [20] to help guide off-the-shelf LLMs in producing re-

executable code from traditional decompilations. Idioms [7] addressed the lack of focus on recovering user-defined composite type definitions, fine-tuning an LLM in a refinement-based approach to encourage generating appropriate user-defined types alongside decompilations. DeGPT [21] deviated from the one-shot LLM paradigm and proposed a multi-agent framework with three distinct, cooperating roles: a referee that proposes an optimization plan to improve the decompilation, an advisor that suggests concrete changes based on the plan, and an operator that verifies the preservation of function semantics. DecLLM [22] similarly departed the one-shot paradigm, iteratively prompting off-the-shelf LLMs to repair decompiler output by using compiler error messages and dynamic runtime feedback as verifiers and LLM context, targeting re-compilability as the primary metric of success.

### B. Reverse Engineering High-Level Languages

There has been some, though limited, existing work on reverse engineering binaries produced by high-level languages like Rust [23], [24] and Swift [25], [26]. The reverse engineering community, however, has produced by far the largest collection of tools and techniques for Go, compared to other modern compiled languages, largely due to its growing presence in cloud and malware programs. Here, we briefly overview relevant Go compilation details, as well as the SOTA in reverse engineering Go.

The Go compiler embeds rich metadata in its binaries, most notably the `pclntab` (program counter line table). The `pclntab` maps addresses to function names and source file information—this persists even in stripped binaries that lack a `.symtab` symbol table as it is required by the Go runtime. For context, a stripped C/C++ binary would not contain any function names at all. A `moduledata` structure describes the layout of the executable to support runtime capabilities like garbage collection and reflection, and since Go 1.18, a `buildinfo` table describes compiler flags, target platform, dependency versions, and other relevant build-time metadata. [27]

These structures have enabled the development of several Go reverse engineering tools. GoReSym [28] parses `pclntab` and `moduledata` to recover function names, type information, and compiler metadata from binaries. The Go Reverse Engineering Tool Kit [29] provides a Go library (GoRE) and a standalone analyzer (Redress) for extracting similar metadata. Plugins also exist for all major disassemblers: AlphaGolang [30] and IDAGolangHelper [31] for IDA Pro [4] and several similar plugins for Ghidra [3] and Binary Ninja [32]. The recently released IDA Pro 9.2 (September 2025) even possesses first-class Go decompilation support [33]. There also exists a limited selection of academic works that discuss reverse engineering Go binaries [34]. Notably, none of these tools and techniques are designed to recover the original Go source code from the binary, and instead merely intend to assist a reverse engineer’s understanding of the binary or improve the quality of decompiled code.

To our knowledge, there is only one prior work that leveraged LLMs to recover the source code of binaries produced by high-level languages. Beyond The C [35] designed and trained

a fine-tuned transformer model in order to allow assembly to be lifted into arbitrary languages, including OCaml, Rust, and Go. Their work involves training custom high and low-level tokenization models for each language they wish to output in, and then using a fine-tuned transformer model to attempt reconstruction of original binaries based on decompiled outputs. This work is somewhat similar to what we attempt; however, we rely instead on off-the-shelf decompilation, replace the fine-tuned transformer model with a SOTA LLM (Gemini 3.1 Flash-Lite), and design a refinement-based approach, rather than applying the LLM directly to the binary for end-to-end decompilation.

### C. Decompilation Benchmarks

Existing benchmarks for evaluating decompiler output focus almost exclusively on C programs targeting x86-64 architecture. First, note that, beyond readability and semantic similarity, *re-compilability* and *re-executability* are common metrics of evaluation in past work. Re-compilability describes whether or not the LLM-generated code can be compiled into an actual binary, while re-executability describes whether or not the LLM-generated code behaves like the original source code.

HumanEval-Decompile [5] is composed of 164 C-transpilation of the small, individual functions from OpenAI’s HumanEval code-generation benchmark [36], and it scores decompiled output primarily by testing if it recompiles under GCC and passes the original assertions. ExeBench [37] is a large-scale dataset of 4.5 million compilable (700,000 executable) real-world C functions sampled from GitHub repositories, each accompanied by automatically generated IO examples. It was originally designed for usage in ML for program optimization and compilers, but decompilation works such as LLM4Decompile have adopted it as an evaluation benchmark for re-executability. DecompileBench [38] comprises 23,400 functions sampled from 130 real-world programs sourced via OSS-Fuzz, and evaluates decompilers along three axes: re-compilability, behavioral correctness via fuzzing-based coverage equivalence, and readability via LLM-as-a-Judge. Finally, Decompile-Bench [39] provides a large training corpus of 2 million function pairs sampled from GitHub repositories and a separate evaluation dataset consisting of only code published after 2025 to minimize data leakage; it evaluates re-executability, edit similarity, and readability.

Beyond The C [35] also constructed a dataset of Go programs for evaluation. However, their dataset consists largely of interview-style coding problems alongside a few other small repositories and, to our knowledge, they did not publicly release their dataset. In contrast, our benchmark is composed entirely of real-world, actively maintained projects to more accurately capture the types of binaries reverse engineers may encounter. Additionally, we have published our dataset on HuggingFace for public use.

Like previous works, we evaluate semantic similarity via methods such as CodeBLEU and LLM-as-a-Judge. However, we do *not* measure re-compilability or re-executability metrics, due to limitations inherent to Go and our design choices;

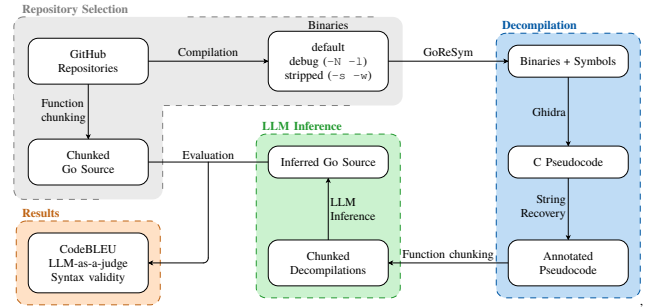


Fig. 1. Diagram of End-to-End Pipeline

we detail our rationale further in Section V.C. Instead, we compromise by measuring the syntactical validity of the decompilations as a rough approximation of re-compilability.

## III. METHODOLOGY

### A. Repository Selection and Compilation

We construct a large-scale benchmark for evaluating Go binary decompilation by scraping the most popular open-source Go repositories hosted on GitHub. Using the GitHub Search API [40], we query for non-archived, non-fork repositories with the Go language tag and at least 500 stars, sorted by star count. We clone the top 200, which provides a corpus of mature projects sufficiently diverse in size and domain. For each repository, we identify all main packages and compile each into a statically linked linux/amd64 binary. We also build three variants per binary to evaluate how compilation flags affect the decompilation quality.

- **default:** Default settings, no flags  
(go build)
- **debug:** Disables optimizations and inlining  
(-gcflags="-N -l")
- **stripped:** Debug symbols and DWARF info removed  
(-ldflags="-s -w")

This yields ~1,800 binaries per variant.

### B. Decompilation

We utilize Ghidra’s [3] headless decompiler via PyGhidra to decompile each binary. Prior to Ghidra’s standard analysis, we run GoReSym [28]: a Go reverse engineering tool that, among other functionalities, recovers function symbol names and locations by parsing the `pcIntab`. This is run on all binaries, not just stripped binaries, since GoReSym reliably recovers function names and locations on all tested binaries; in contrast, Ghidra’s more limited Go analysis support fails for some functions.

Each Go binary includes all necessary functions for execution: runtime-related, standard library, external library, and user-defined functions. But, as these are duplicated many times across different binaries, analyzing every single appearance of common runtime or library functions would result in these functions dominating the results. Moreover, these functions are of limited decompilation interest—the Go runtime and Go standard library are open-source and well-analyzed, and tools like IDA’s Lumina [41] also allow teams to store common function metadata for recognition across binaries.

Thus, we limit our scope to only user-authored code, i.e. code owned by the target repository.

For this purpose, we extract the module path from each binary with `go version -m` and decompile only functions whose name matches `main.*` or whose name prefix matches the module path, which is a link to the Github repo such as `github.com/antonmedv/fx`. The first filters for functions in the main package of the binary, while the second filters for function names like `github_com_antonmedv_fx_internal_toml_toJ`, which are functions defined in non-main packages. (In this instance, this function `toJ` is defined in the package `toml` in the file `internal/toml/toml.go`) This reduces the size of the subset of the binary that is decompiled by 80–99%, substantially reducing computation time and evaluation noise.

Some extremely large functions take very long to decompile, as the time to decompile grows superlinearly with respect to the function size; thus, we add a 10-minute timeout for each function. Only [UPDATE LATER]% of the total corpus of functions are filtered out by this timeout.

### C. String Recovery

In Go binaries, strings are not null-terminated, and are instead placed in the binary in order of length and concatenated together into much larger *packed string blocks* [42]. Then, in

the program itself, strings are internally represented as 16-byte (`pointer`, `length`) pairs, where `pointer` points to somewhere within the larger strings in the `.rodata` of the binary. The Go runtime then extracts the desired string from the packed blocks when necessary. Ghidra, however, expects strings as null-terminated C strings, and does not typically include the strings within the function decompilation itself (occasionally, it does include a portion of the packed blocks after resolving a pointer, which may contain as a substring the desired string). Instead, Ghidra’s auto-analysis typically labels these string structures with symbols such as `s_*`, `PTR_s_*`, and `DAT_*`. Without access to the binary itself, it’s necessary to resolve the string literals themselves; otherwise, the LLM would see the strings as merely opaque memory references. Thus, ComeBack utilizes some manually-designed heuristics to recover some of the original strings.

- 1) **Defined Data Scan:** We first scan the data defined by Ghidra for two categories of labels.
  - a) A symbol named `s_*` points directly to the start of the string in the packed block. We can use Ghidra’s data API to directly retrieve the bytes at this location for up to some maximum number of bytes, since we lack the length metadata of the string. Empirically, we use a maximum of 200 bytes; this read may overflow into adjacent strings, but it usually retains the desired string without introducing too much other noise.
  - b) A symbol named `PTR_*` *may* represent a pointer to a pointer to a string literal. In other words, the quadword at address `PTR_*` may point to the start of the string in a packed block. Thus, we attempt to dereference `PTR_*`, and then apply the same strategy as used for `s_*` symbols.
- 2) **Undefined Data Resolution:** The decompiler may also include `PTR_s_<hexstr>` and `s_<hexstr>` symbols in the pseudocode that are not defined as data items in the Ghidra API listing. We search for such references, parse the hex address suffix, and resolve the memory address—we attempt to read an array of string structs that commonly appear as a result of string tables and slice initializers. For each potential string struct (`pointer`, `length`), we read exactly `length` bytes from `pointer`. If array reading fails, we fallback to the simple string resolution described in 1).
- 3) **DAT Symbols:** For data Ghidra cannot classify, it frequently labels it as `DAT_<hexstr>`. For such symbols, we parse the hex address, interpreting it as the pointer to the string, and search the adjacent decompiled C code for a corresponding `length` value. This is because the stack often contains string structs, and they show up in Ghidra decompilations as, e.g., `local_38 = DAT_xxx`; `local_30 = 0xd`; . We fallback to 1) if this fails.
- 4) **Hex Literals:** There may also exist raw hex literals in the decompiled code. For such literals, we check if they

#### Heuristic 1: Defined Data Scan

Scans Ghidra’s typed items once per binary. `s_*` labels yield strings directly; `PTR_s_*` labels are pointer-dereferenced.

```
From tailscale/tailscale - client/systray.setAppIcon (stripped build):
// Strings:
// (@s_loading_00f87a900) = "loading"
C pseudocode - recovered string used in comparison:
if ((in_stack_00000008 == s_loading_00f87a900)
    && (DAT_00f7a908 == cStack0000000000000010)) {
    tailscale_com_client_systray_startLoadingAnimation();
}
Ghidra typed s_loading as string data at 0xf7a900; the pipeline reads "loading" directly from that address.
```

#### Heuristic 2: Undefined Data Resolution

Resolves `PTR_s_*` / `s_*` references in C pseudocode absent from the data listing. Parses hex suffix from name; reads `Go(ptr, len)` pairs for precise, length-delimited extraction.

```
From usememos/memos - plugin/filter.NewAttachmentSchema (default build):
// Strings:
// (@PTR_s_attachmentindex_html...01aad1700) = ["attachment", "memo_id"]
// (@PTR_s_attachmentindex_html...01aacfc00) = ["attachment", "filename",
// "mime_type", "scalar", "string", "attachment", "type",
// "create_time", "scalar", "timestamp", ...]
C pseudocode referencing the symbols (initializing map entries):
local_68 = PTR_s_attachmentindex_html...01aacfc0;
uStack_60 = _UNK_01aacfc0;
runtime_mapassign(in_RDI, in_RSI, key, &PTR_DAT_01a76720);
Hex suffix 01aacfc0 parsed -> memory at 0x1aacfc0 read as Go(ptr, len) pairs -> 17 domain-specific strings recovered.
```

#### Heuristic 3: DAT Symbols

Resolves `DAT_*` labels (unclassified data). Detects paired string lengths from adjacent stack locals or function arguments.

```
From ollama/ollama - main.main.func1 in pull-progress (default build):
// Strings:
// (@DAT_007ac68b0) = "Progress: status=vv, total=vv, completed=vv"
C pseudocode - used as a format string argument:
w.tab = (internal_abi_ITab *)0x2c; //length = 44
format.len = (int)&DAT_007ac68b;
format.str = extracout_RDX;
fmt_printf(w, format, in_stack, 3, ...);
Hex address parsed from label. Length 0x2c (44) detected from adjacent assignment, matching the 44-byte format string.
```

#### Heuristic 4: Hex Literals

Resolves bare hex constants in C pseudocode that fall within `.rodata` address ranges.

```
From argoproj/argo-workflows - v1alpha1.ClusterWorkflowTemplate.GetResourceScope (stripped build):
// Strings:
// (@0x31710320) = "cluster"
C pseudocode - bare hex address returned as a string pointer:
undefined8 ...GetResourceScope(void) {
    return 0x3171032;
}
Address 0x3171032 falls within .rodata. The decompiler shows only a numeric constant; Heuristic 4 resolves it to "cluster".
```

Listing 1. Examples of string recovery heuristics

point to memory within `.rodata`, and then apply the strategy from 3).

Resolved strings are injected as annotations at the top of each function’s decompiled output, providing the LLM with the likely string literal (or packed string block) that corresponds to each symbol.

#### D. Function-Level Chunking and Ground Truth

Due to limitations in context window size, detailed further in Section V, we divide the decompiled output by function, to perform inference on each individually. In order to recover the original ground truth for comparison, we run `go list -deps -json` for each main package in each repository to identify the source code files in the repository that contribute to this package. Then, we parse each file for function and method declarations, and isolate each declaration and corresponding definition into its own file.

Notably, we organize both the chunked decompilations and ground truth code according to the path to the original source code file in the repository. For the ground truth code, this is trivial; for the decompilation, the symbol names provide the path: the recovered source for `github_com_antonmedv_fx_internal_toml_toJ` is placed in `internal/toml/toJ.go`.

Note that, due to compiler-generated artifacts—separate functions for closures and Goroutines (`Foo.func1`), defer wrappers (`Foo.deferwrap1`), and generic instantiations (`Foo[go.shape.struct_{...}]`)—the decompilation chunking process applies some heuristics to consolidate such artifacts with the source-level parent declaration. This is important because the ground truth source includes these details, and the LLM thus needs this context in order to recover these specific sections of code or to contextualize that, e.g., the function is a generic function.

Our benchmark includes these chunked decompilations and ground truth source code.

#### E. LLM-Based Source Recovery

Finally, we frame source recovery as a code translation task: given a single function’s Ghidra pseudo-C decompilation, an LLM is prompted to recover the corresponding Go source. We provide the following system instruction:

You are an expert Go reverse engineer. You will be given a single decompiled C pseudocode function produced by Ghidra from a compiled Go binary. Recover the original Go source code for this function as accurately as possible. Output ONLY valid Go code with no explanation. Exclude package declarations and imports.

And we include the pseudo-C Ghidra decompilation of a single function in the user message.

## IV. EVALUATION

### A. Metrics

We evaluate recovered functions using three complementary metrics that capture different aspects of decompilation quality:

#### a) CodeBLEU:

CodeBLEU [43] is a composite evaluation metric, originally designed for assessing code generation quality, that measures four aspects of the code: n-gram precision (lexical similarity), weighted n-gram precision (keyword-aware matching), syntax match (abstract syntax tree’s subtree overlap), and dataflow match (variable usage pattern similarity).

#### b) LLM-as-a-Judge:

LLM-as-a-Judge [44] has grown in popularity as an evaluation metric for tasks where quantitative evaluation is hard. Existing literature on decompilers has increasingly been measuring success with LLM-as-a-Judge, by asking an LLM to assess readability, semantic similarity, or other aspects of the decompiled code by comparing it with the original source code [5], [6], [8].

We employ a separate Gemini instance of the same model (Gemini 3.1 Flash-Lite) to evaluate semantic similarity between the inferred decompilation and the original source code, prompting it to rate semantic similarity on an integer scale from 0 (completely different) to 10 (semantically identical). This serves to capture high-level functional correctness that lexical metrics may miss; for instance, a function with different variable names or control flow structure than the source code that implements identical logic should score highly on the LLM-as-a-judge semantic similarity metric even if CodeBLEU penalizes the surface differences.

#### c) Syntax Validity:

We parse each decompilation with `tree-sitter` [45], a programming language parser, and assign a score of 1 for syntactically valid code and a score of 0 for any syntax errors.

#### d) Complexity-Weighted Analysis:

For all metrics, we report both an unweighted mean and a source-length-weighted mean across the evaluated corpus. Moreover, using `tree-sitter` [45], we bin results by:

- AST node count (6 bins: [1,25), [25,50), [50,100), [100,200), [200,500), [500,∞))
- AST depth (6 bins: [1,5), [5,10), [10,15), [15,20), [20,25), [25,∞))

This organization of the results allows us to analyze the relationship between source code complexity and decompilation quality and ensure that the scores of larger, more complex functions are not hidden behind the scores of many small, trivial functions.

### B. Results

Our final dataset comprises 128 repositories and 1,665 unique binaries, yielding approximately 587,000 (default), 594,000 (debug), and 624,000 (stripped) user-authored functions per variant—over 1.8 million function-level evaluations in total.

#### a) Aggregate Scores:

Fig. 2 summarizes the distribution of all three metrics across build variants. The LLM-as-a-Judge semantic similar-

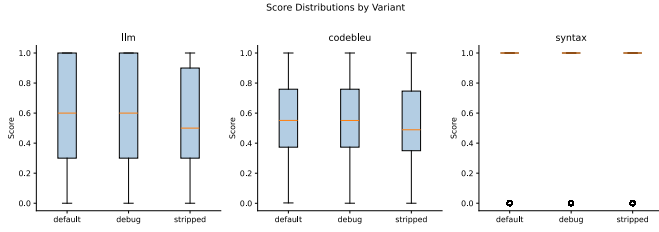


Fig. 2. Score distributions by variant across all three metrics. Boxes span Q1–Q3; whiskers extend to  $1.5 \times \text{IQR}$ .

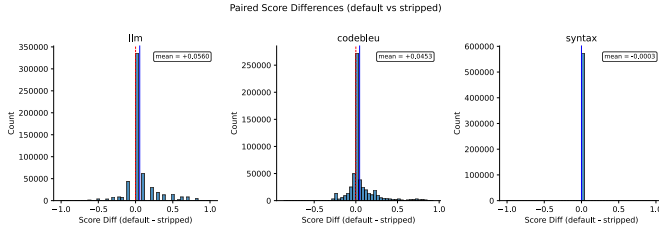


Fig. 3. Distribution of paired score differences (default — stripped) for the same function across build variants.

ity score averages 0.61 for the default and debug variants and 0.55 for stripped, on a 0–10 scale normalized to [0, 1]. CodeBLEU follows a similar pattern, averaging 0.59 (default/debug) and 0.54 (stripped). Syntax validity is near-perfect at 99.6% across all three variants, indicating that the LLM reliably produces parseable Go regardless of build configuration. The source-length-weighted means are substantially lower—0.43 (LLM) and 0.44 (CodeBLEU) for the default variant—reflecting that larger, more complex functions, which contribute more weight, are harder to decompile accurately.

#### b) Effect of Build Variant:

To isolate the effect of compilation flags, we perform paired comparisons between the same functions across build variants. Default and debug builds produce nearly identical scores: the mean paired difference is less than 0.001 for both LLM and CodeBLEU. A Wilcoxon signed-rank test [46] finds the LLM difference statistically significant ( $p = 0.01$ ) but with a negligible effect size (mean difference  $< 0.001$ ), and no significant difference for CodeBLEU ( $p = 0.15$ ). In contrast, stripping debug symbols and DWARF information degrades both metrics significantly ( $p < 10^{-10}$ , Wilcoxon). As shown in Fig. 3, the default variant outperforms stripped by a mean of +0.056 on LLM and +0.045 on CodeBLEU. This is expected: while stripped Go binaries still retain the `pcntab` critical for GoReSym and Ghidra analysis, their lack of symbol-table information still negatively impacts type and variable recovery, forcing the LLM to work with a less informative decompilation. Interestingly, syntax validity is marginally *higher* for stripped builds (by 0.03 percentage points), though this difference is negligible in practical terms.

#### c) Complexity Analysis:

Both semantic similarity metrics exhibit a strong inverse relationship with source code complexity. Fig. 4 plots mean scores against ground-truth AST node count, while Fig. 5 plots mean scores against ground-truth AST tree depth. Functions with 25–50 AST nodes score highest (0.80 LLM,

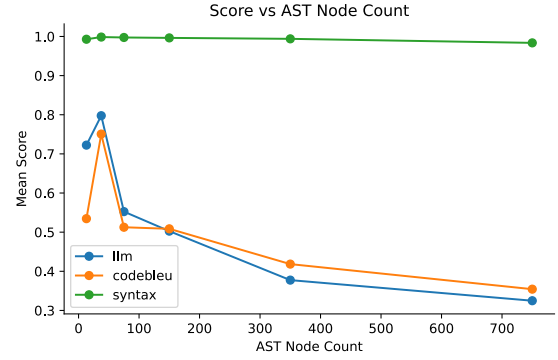


Fig. 4. Mean scores by ground-truth AST node count. LLM and CodeBLEU degrade substantially with complexity; syntax validity remains high.

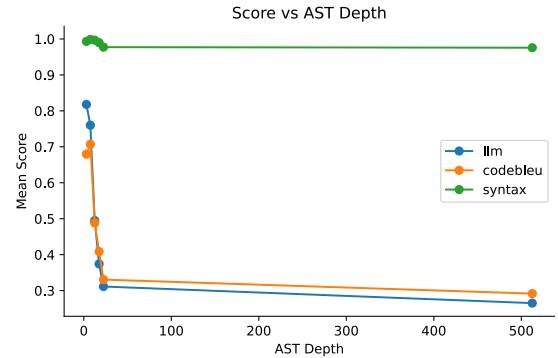


Fig. 5. Mean scores by ground-truth AST tree depth. LLM and CodeBLEU degrade substantially with complexity; syntax validity remains high.

0.75 CodeBLEU), while functions exceeding 500 nodes score roughly half as well (0.32 LLM, 0.35 CodeBLEU). The relationship with AST depth is similar: functions of depth 5–10 average 0.76 (LLM) and 0.71 (CodeBLEU), declining to 0.27 and 0.29 at depths above 25. This is consistent with the Spearman correlations of  $\rho = -0.51$  (AST node count vs. LLM) and  $\rho = -0.50$  (AST node count vs. CodeBLEU). Notably, the smallest functions ([1, 25) nodes) score slightly lower than the [25, 50) bin; these are typically trivial getter/setter or stub functions where the ground truth is a single return statement, and the LLM may over-generate. Syntax validity remains high across all bins, dipping only to 98.4% for functions above 500 nodes.

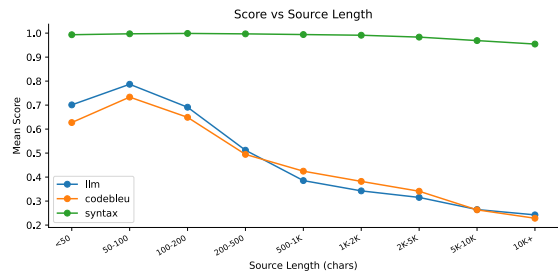


Fig. 6. Mean scores by source length (characters). The strong negative trend for LLM and CodeBLEU mirrors the AST-complexity results; syntax validity is largely unaffected.



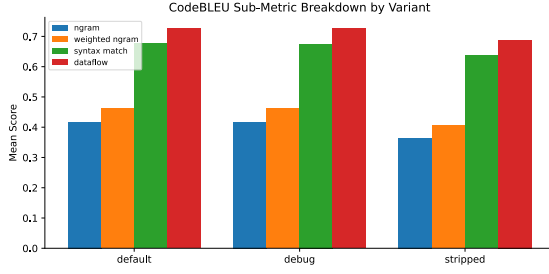


Fig. 7. CodeBLEU sub-metric breakdown by variant. Structural metrics (syntax match, dataflow) score higher than lexical metrics (n-gram, weighted n-gram).

Fig. 6 corroborates the AST-complexity findings from a different angle: binning by raw source length (in characters) yields a similarly strong negative trend for both LLM ( $\rho = -0.51$ ) and CodeBLEU ( $\rho = -0.50$ ), while syntax validity remains nearly flat.

#### d) CodeBLEU Sub-Metric Breakdown:

Decomposing CodeBLEU into its four sub-metrics (Fig. 7) reveals that structural similarity is recovered more effectively than lexical similarity. Dataflow match scores highest (0.73 for default/debug, 0.69 for stripped), followed by syntax match (0.68, 0.64), weighted n-gram (0.46, 0.41), and n-gram (0.42, 0.36). This pattern suggests that the LLM successfully reconstructs the high-level control flow and variable usage patterns of the original Go code, even when exact token sequences differ—likely due to renamed variables, reordered statements, or alternative but semantically equivalent expressions. The n-gram metrics, which penalize such surface-level differences, are correspondingly lower.

#### e) Metric Agreement:

The LLM-as-a-Judge and CodeBLEU metrics are strongly correlated (Pearson  $r = 0.77$ , Spearman  $\rho = 0.76$ ; Fig. 8), lending confidence that the LLM-as-a-Judge scores accurately

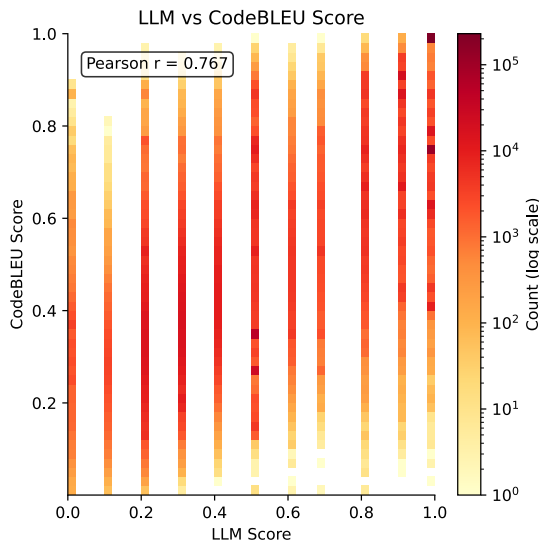


Fig. 8. Joint distribution of LLM-as-a-Judge and CodeBLEU scores across all evaluated functions. The strong positive correlation (Pearson  $r = 0.77$ ) indicates that both metrics capture similar underlying quality.

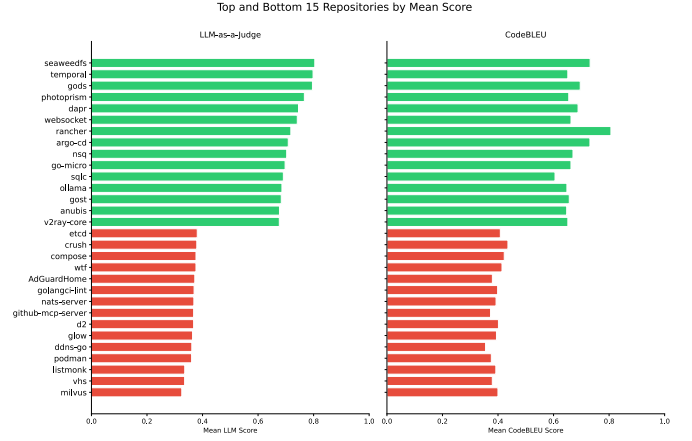


Fig. 9. Mean LLM scores for the top and bottom 15 repositories (minimum 20 functions). Substantial inter-repository variation exists beyond what variant or complexity alone explains.

capture semantic similarity. Both metrics also show moderate negative correlation with source length (Spearman  $\rho \approx -0.50$ ), confirming the complexity trend observed in the binned analysis.

#### f) Per-Repository Variation:

Aggregate statistics mask considerable variation across repositories. Fig. 9 shows the top and bottom 15 repositories ranked by mean LLM score. The best-performing repositories achieve mean scores above 0.8, while the worst fall below 0.4—a spread far larger than the variant effect (+0.056). This variation likely reflects differences in coding style, function granularity, and the extent to which idiomatic Go patterns are preserved through compilation and decompilation.

## V. DISCUSSION

### A. Dataset Contamination

Practically all SOTA LLMs are trained on mass amounts of data scraped from the internet. Considering the recent focus on the application of LLMs to coding tasks [47], [48], [49], it is likely that a significant proportion of the most popular Go repositories on GitHub, which comprise our dataset, were seen by LLMs during training. This contamination may induce some systematic error in our evaluations; unfortunately, due to limitations in time and resources, it is difficult to recover a large corpus of Golang binaries and corresponding sources outside of existing open source repositories to create an uncontaminated dataset. We leave it to future work to determine the extent to which data contamination inflates our metrics of success.

### B. Context Window Size

The Gemini API offers inference with models possessing context windows of up to  $\sim 1\text{M}$  tokens. However, the binaries produced from Go programs are large, due to extensive static linking of the Go standard library and external packages—after tokenization, they frequently exceeded the context window limits by 1-2 orders of magnitude. This precluded us from designing or evaluating an end-to-end decompilation pipeline for the programs. Moreover, the decompilations produced by

Ghidra are even larger than the binaries themselves; hence, we chose to divide each decompilation into smaller chunks that fit within the context window limit, and perform inference separately. Unfortunately, this restricts the LLM from considering information across the entire decompilation, some of which, e.g. parent/child functions in call graph or methods of the same object, may be critical for accurate reconstruction.

We also found that saturating the context window with requests between 100,000 and 1M input tokens frequently resulted in nonsensically simple outputs with size well below the output token limit. Empirically, isolating each function by itself in the context window allowed the LLM to focus on just the function and produce more coherent outputs; yet, as aforementioned, this restricts the LLM’s source of information to merely the decompilation of the target function. This guided our decision to limit the Go source recovery to only functions. While function and control flow recovery is arguably the most critical task of decompilation, the recovery of object structures, imports, and other relevant information may also be desirable. However, without global or cross-function context, recovering such facets of the program becomes substantially more difficult. We hope future work may explore both alternative chunking strategies and the recovery of supplemental information.

### C. Re-compilability and Re-executability

Existing literature in LLM-based decompilation often benchmarks LLM output against re-compilability and re-executability metrics [5], [6], [8], [9], [39]. These benchmarks are composed of individual, standalone C functions that compile in isolation with only standard library dependencies, and the LLM receives the entire function’s decompilation as input. Re-executability evaluation relies on either manually designed test cases, as in HumanEval-Decompile, which limits scalability, or automatically generated IO examples, as in ExeBench, which may not comprehensively test correctness.

Such an evaluation paradigm is difficult to replicate for the complex Go projects we include in our benchmark. Unlike C, where a single function with standard includes can compile independently, Go requires a valid package declaration, correct imports, and type-consistent definitions across files. Moreover, because Go binary decompilations far exceed LLM context limits, our pipeline chunks by function—the LLM never sees the full program and therefore cannot produce the package structure, import statements, or cross-file type definitions required for compilation. Re-compiling each Go program from individually decompiled functions would require solving a program synthesis problem orthogonal to decompilation quality. We therefore evaluate our system on neither re-compilability nor re-executability, and instead assess syntactic correctness of each function as a practical proxy, alongside CodeBLEU and LLM-as-a-Judge for semantic similarity.

### D. String Recovery

Our string recovery heuristics, while effective at surfacing many string literals that would otherwise appear as opaque memory references, have several notable limitations. Phase 1’s scan-until-non-printable strategy reads up to 200 bytes from a

pointer without knowing the true string length, and frequently over-reads into adjacent strings in the packed `.rodata` block. The LLM thus receives noise concatenated with the actual string and must infer the boundary on its own, which can degrade accuracy. Phases 3 and 4 attempt to mitigate this by detecting paired lengths from adjacent assignments in the decompiled code, but this heuristic is pattern-dependent and fails when the decompiler reorders or optimizes away the length assignment. Moreover, a non-negligible proportion of string references go unresolved: hex literals falling outside `.rodata`, strings constructed dynamically at runtime (e.g. via `fmt.Sprintf`), and symbols that do not conform to Ghidra’s `s_*`, `PTR_s_*`, or `DAT_*` naming conventions are missed entirely. We did not measure an overall string recovery rate across the corpus, and leave such quantification to future work.

Unresolved or noisy strings can directly degrade downstream LLM output. For instance, a missing format string may significantly decrease the likelihood of the LLM recovering an `fmt.Sprintf` or `fmt.Errorf` call; a 200-byte packed block annotation where only 7 bytes are relevant may cause the LLM to hallucinate extra string content. Finally, commercial tools such as IDA Pro’s GoStrings plugin [50] offer more robust Go string recovery, but are restricted behind a paywall; access to such tools may improve the quality of string annotations provided to the LLM.

### E. Coding Agents

Recent LLM-based coding agents have shown strong results on programming benchmarks [51], [52] and programming-adjacent benchmarks, e.g. cybersecurity [53]. To our knowledge, past work on LLM-based decompilers has not considered the use of such coding agents. We hope to explore the application of such agents in the future, especially considering their demonstrated capabilities working in code repositories with sizes much larger than their available context windows. For instance, a coding agent’s global view of the binary and decompilation, although potentially narrow relative to the total content, could prove crucial for LLM-based recovery of supplemental information and strings. Their strong coding abilities may also be helpful for enabling re-compilability.

### F. Compiler Optimizations

Some evaluation metrics are erroneously low since the Go compiler inlines several functions for optimization, and the evaluation metrics cannot easily note the similarity between a function call and its inlined version, even if the recovery of the function’s logic is perfect. We believe its impact is largely negligible, though, since the inlining process impacts a sufficiently small proportion of the total corpus of functions for evaluation.

Other compiler optimizations, such as constant folding or common subexpression elimination, may also affect evaluation in unexpected ways. Nevertheless, we believe such optimizations impact a negligible fraction of our dataset.



### G. Shared Functions

Many repositories produced more than one binary. For each such repository, their binaries often included the same user-defined functions, typically internal library functions. For the purpose of evaluating each binary in isolation, i.e. to simulate the typical situation where a reverse engineer may gain access to only a single binary, our system applied the LLM to each copy of this function separately. This may slightly bias aggregate metrics toward internal library functions that is frequently shared by different binaries, but we hypothesize that the impact is negligible, considering the large corpus size and that most repositories have merely 1-10 binaries.

As a result of the non-determinism of both the compilation and decompilation process (with regards to a single function as a part of a larger program), the Ghidra decompilations of the same function may differ between each binary. Thus, it may be useful to consider how providing the different Ghidra decompilations as context impacts the LLM’s recovery accuracy. We leave such questions to future work.

### H. Excluding Imports

We instruct the LLM, in the system prompt, to exclude package declarations and import statements from its output. Since our pipeline performs inference on individual functions in isolation, the LLM can often lack the cross-function and cross-file context necessary to reliably reconstruct a complete and correct import block. And, importantly, our chunked ground truth source code also excludes imports, as each ground truth file contains only the body of a single function or method declaration. Including imports in the LLM output would introduce tokens absent from the ground truth, diluting lexical similarity metrics—particularly for short functions, where a few extraneous import lines could cause substantial reductions in CodeBLEU scores. We acknowledge that import recovery is a meaningful aspect of full program reconstruction, as import paths carry key semantic information (e.g. `context.Context`, `sync.Mutex`) that signals the abstractions in use and contextualizes a function’s purpose in the entire program. However, recovering imports is fundamentally a whole-program task that requires knowledge of all types referenced across the entire package, and is thus outside the scope of our per-function evaluation. We leave the recovery of import statements to future work.

### I. Obfuscation

Our dataset does not include any obfuscated binaries (e.g. obfuscated `pcntab`) and many of *ComeBack*’s heuristics, such as function name recovery and string recovery, may fail when applied to obfuscated Go binaries. In particular, this limits our method’s applicability to Go malware, which is often obfuscated.

### J. Inference vs. Fine-Tuning

Several previous works [5], [6], [8], [9], [35] fine-tune their own models for decompilation, while others [11], [21] use existing models for inference. The detriment of no fine-tuning is that our method loses out on the demonstrated benefits of fine-tuning for the task-specific domain [54];

nonetheless, our approach offers more flexibility to researchers and reverse engineers, as SOTA LLMs like Gemini, Claude, and GPT can be incorporated into our method without any prior modification [11]. This is particularly important considering that recent benchmarks [38] demonstrate that SOTA LLMs such as GPT-4o outperform fine-tuned approaches like LLM4Decompile [5] and MLM [55], largely due to the rapid improvements observed in SOTA LLMs.

### K. Costs

The inference costs of this project totalled to about \$1,100 in Gemini inference credits. Additionally, we were unable to perform comparisons between our pipeline with Gemini versus other SOTA models, both proprietary and open-source, due to limited resources.

## VI. CONCLUSION

We created a novel, real-world benchmark for assessing decompilers’ capabilities to recover the original Go source code from a Go binary and proposed *ComeBack*, an end-to-end LLM-based decompilation pipeline to reconstruct Go source code. Through qualitative and quantitative evaluation on our dataset, we demonstrate that *ComeBack* is effective at producing semantically similar, readable Go code to assist the reverse engineering of Go binaries. We will be publishing our dataset to HuggingFace and have open-sourced *ComeBack* on GitHub<sup>1</sup> for public use.

## ACKNOWLEDGEMENTS

We thank Google for sponsoring Gemini inference credits for this project. We also thank David Wagner (UC Berkeley) for his advice and guidance.

## REFERENCES

- [1] E. J. Schwartz, J. Lee, M. Woo, and D. Brumley, “Native x86 decompilation using semantics-preserving structural analysis and iterative control-flow structuring,” in *Proceedings of the 22nd USENIX Conference on Security*, in SEC’13. Washington, D.C.: USENIX Association, 2013, pp. 353–368.
- [2] M. Noonan, A. Loginov, and D. Cok, “Polymorphic Type Inference for Machine Code.” [Online]. Available: <https://arxiv.org/abs/1603.05495>
- [3] National Security Agency, “Ghidra Software Reverse Engineering Framework.” [Online]. Available: <https://github.com/NationalSecurityAgency/ghidra>
- [4] Hex-Rays, “IDA Pro Disassembler and Debugger.” [Online]. Available: <https://hex-rays.com/ida-pro>
- [5] H. Tan, Q. Luo, J. Li, and Y. Zhang, “LLM4Decompile: Decompiling Binary Code with Large Language Models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2024, pp. 3473–3487. doi: 10.18653/v1/2024.emnlp-main.203.
- [6] Y. Feng, B. Li, X. Shi, Q. Zhu, and W. Che, “ReF Decompile: Relabeling and Function Call Enhanced Decompile.” [Online]. Available: <https://arxiv.org/abs/2502.12221>
- [7] L. Dramko, C. L. Goues, and E. J. Schwartz, “Idioms: Neural Decompile With Joint Code and Type Definition Prediction.” [Online]. Available: <https://arxiv.org/abs/2502.04536>

<sup>1</sup><https://github.com/nightxade/come-back>

- [8] H. Tan *et al.*, “SK2Decompile: LLM-based Two-Phase Binary Decompile from Skeleton to Skin.” [Online]. Available: <https://arxiv.org/abs/2509.22114>
- [9] Y. Wang, X. Xu, X. Zhu, X. Gu, and B. Shen, “SALT4Decompile: Inferring Source-level Abstract Logic Tree for LLM-Based Binary Decompile.” [Online]. Available: <https://arxiv.org/abs/2509.14646>
- [10] X. She, Y. Zhao, and H. Wang, “WaDec: Decompiling WebAssembly Using Large Language Model.” [Online]. Available: <https://arxiv.org/abs/2406.11346>
- [11] X. Wang, Y. Hu, and K. Leach, “Context-Guided Decompile: A Step Towards Re-executability.” [Online]. Available: <https://arxiv.org/abs/2511.01763>
- [12] W. Fang, Z. Zhou, J. He, and W. Wang, “StackSight: Unveiling WebAssembly through Large Language Models and Neurosymbolic Chain-of-Thought Decompile.” [Online]. Available: <https://arxiv.org/abs/2406.04568>
- [13] Z. Liao *et al.*, “Augmenting Smart Contract Decompiler Output through Fine-grained Dependency Analysis and LLM-facilitated Semantic Recovery.” [Online]. Available: <https://arxiv.org/abs/2501.08670>
- [14] The Go Authors, “The Go Programming Language.” [Online]. Available: <https://golang.org/>
- [15] Go, “Go for Cloud & Network Services.” [Online]. Available: <https://go.dev/solutions/cloud>
- [16] unixpickle, “gobfuscate.” [Online]. Available: <https://github.com/unixpickle/gobfuscate>
- [17] P. A. N. Unit 42, “The Gopher in the Room: Analysis of GoLang Malware in the Wild.” [Online]. Available: <https://unit42.paloaltonetworks.com/the-gopher-in-the-room-analysis-of-golang-malware-in-the-wild/>
- [18] CrowdStrike, “Golang Malware Is More than a Fad: Financial Motivation Drives Adoption.” [Online]. Available: <https://www.crowdstrike.com/en-us/blog/financial-motivation-drives-golang-malware-adoption/>
- [19] N. Harrand, C. Soto-Valero, M. Monperrus, and B. Baudry, “The Strengths and Behavioral Quirks of Java Bytecode Decompilers,” in *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, IEEE, 2019, pp. 92–102. doi: 10.1109/scam.2019.00019.
- [20] Q. Dong *et al.*, “A Survey on In-context Learning.” [Online]. Available: <https://arxiv.org/abs/2301.00234>
- [21] W. K. Wong *et al.*, “Refining Decompiled C Code with Large Language Models.” [Online]. Available: <https://arxiv.org/abs/2310.06530>
- [22] W. K. Wong *et al.*, “DecLLM: LLM-Augmented Reversible Decompile for Enabling Programmatic Use of Decompiled Code.” [Online]. Available: <https://doi.org/10.1145/3728958>
- [23] Z. Zhou, “Decompiling Rust: An Empirical Study of Compiler Optimizations and Reverse Engineering Challenges.” [Online]. Available: <https://arxiv.org/abs/2507.18792>
- [24] C. P. Research, “Rust Binary Analysis, Feature by Feature.” [Online]. Available: <https://research.checkpoint.com/2023/rust-binary-analysis-feature-by-feature/>
- [25] doronz88, “Reversing Swift.” [Online]. Available: [https://github.com/doronz88/swift\\_reversing](https://github.com/doronz88/swift_reversing)
- [26] M. Kraus and V. Hupert, “The Swift Language from a Reverse Engineering Perspective,” in *Proceedings of the 2nd Reversing and Offensive-Oriented Trends Symposium*, in ROOTS '18. Vienna, Austria: Association for Computing Machinery, 2018. doi: 10.1145/3289595.3289599.
- [27] 0xdealias, “Reverse Engineering Golang.” [Online]. Available: <https://gist.github.com/0xdealias/4e430914124c3fd2c51cb7ac2801acba>
- [28] Mandiant, “GoReSym.” [Online]. Available: <https://github.com/mandiant/GoReSym>
- [29] goret, “Go Reverse Engineering Tool Kit.” [Online]. Available: <https://github.com/goret>
- [30] S. Labs, “AlphaGolang.” [Online]. Available: <https://github.com/SentineLabs/AlphaGolang>
- [31] sibears, “IDAGolangHelper.” [Online]. Available: <https://github.com/sibears/IDAGolangHelper>
- [32] Vector35, “Binary Ninja.” [Online]. Available: <https://binary.ninja/>
- [33] Hex-Rays, “Stop Guessing and Start GOing: Cleaner Golang Decompile in IDA 9.2.” [Online]. Available: <https://hex-rays.com/blog/stop-guessing-and-start-going>
- [34] A. Engelke, “Reconstructing Program Semantics from Go Binaries,” Master’s thesis, 2017. [Online]. Available: <https://home.cit.tum.de/~engelke/pubs/1709-ma.pdf>
- [35] I. Hosseini and B. Dolan-Gavitt, “Beyond the C: Retargetable Decompile using Neural Machine Translation,” in *Proceedings 2022 Workshop on Binary Analysis Research*, in BAR 2022. Internet Society, 2022. doi: 10.14722/bar.2022.23009.
- [36] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code.” [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [37] J. Armengol-Estapé, J. Woodruff, A. Brauckmann, J. W. d. S. Magalhães, and M. F. P. O’Boyle, “ExeBench: an ML-scale dataset of executable C functions,” in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, in MAPS 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 50–59. doi: 10.1145/3520312.3534867.
- [38] Z. Gao *et al.*, “DecompileBench: A Comprehensive Benchmark for Evaluating Decompilers in Real-World Scenarios.” [Online]. Available: <https://arxiv.org/abs/2505.11340>
- [39] H. Tan *et al.*, “Decompile-Bench: Million-Scale Binary-Source Function Pairs for Real-World Binary Decompile.” [Online]. Available: <https://arxiv.org/abs/2505.12668>
- [40] Github, “Github Search API.” [Online]. Available: <https://docs.github.com/en/rest/search?apiVersion=2026-03-10>
- [41] Hex-Rays, “Private Lumina.” [Online]. Available: <https://hex-rays.com/lumina>
- [42] T. Mathison, “Golang Reverse Engineering Tips.” [Online]. Available: <https://www.travismathison.com/posts/Golang-Reverse-Engineering-Tips/>
- [43] S. Ren *et al.*, “CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.” [Online]. Available: <https://arxiv.org/abs/2009.10297>
- [44] J. Gu *et al.*, “A Survey on LLM-as-a-Judge.” [Online]. Available: <https://arxiv.org/abs/2411.15594>
- [45] tree-sitter, “Go Tree-sitter.” [Online]. Available: <https://github.com/tree-sitter/tree-sitter>
- [46] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [47] Anthropic, “Claude Code | Anthropic’s agentic coding system.” [Online]. Available: <https://www.anthropic.com/product/claude-code>
- [48] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code.” [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [49] Google, “Gemini CLI: your open-source AI agent.” [Online]. Available: <https://blog.google/innovation-and-ai/technology/developers-tools/introducing-gemini-cli-open-source-ai-agent/>
- [50] Oxjiayu, “Yet Another Golang binary parser for IDAPro.” [Online]. Available: [https://github.com/Oxjiayu/go\\_parser](https://github.com/Oxjiayu/go_parser)
- [51] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?.” [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [52] M. A. Merrill *et al.*, “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces.” [Online]. Available: <https://arxiv.org/abs/2601.11868>
- [53] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, “CyberGym: Evaluating AI Agents’ Real-World Cybersecurity Capabilities at Scale.” [Online]. Available: <https://arxiv.org/abs/2506.02548>
- [54] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models.” [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [55] AscendGrace, “MLM.” [Online]. Available: <https://mlm01.com/index>